Xijing Wang (Thomas)
Team 6
Teammates: Dillon Ma, Mithun Roy, Leena Aljurashi
IRL01
Jan 29, 2026

Individual Process

This week, I worked on both the sensor lab and mock-up activities with my team. For the sensor lab, my primary contribution was implementing physical units and semantic interpretation of sensor outputs. I fixed the code to ensure all sensor readings were displayed in meaningful, human-readable physical units instead of raw voltages or binary/numerical values.

Specifically, I focused on processing and interpreting data from the IMU, potential meter and joystick. Raw IMU outputs values with 0-360 (direction degrees), and I use a function to transfer into carnival directions such as North, East, South, West based on angular ranges. Potentiometer outputs, from 0-1023, were mapped to a speed level from 1 to 10 with around 100 threshold. Joystick has X, Y values which can be interpreted as movement directions such as moving forward/backward, forward/backward-right/left, stationary. In addition, the joystick button state was translated into a pause state, with true/false values. These conversion allowed that all sensor outputs were presented in physical or semantic units can directly correspond to robot behavior.

While Mithun was primarily responsible for the hardware setup, wiring and low-level sensor testing, I implemented the logic responsible for unit conversion, semantic mapping and final output formatting. Due to severe snow conditions during the first week, other team members were unable to access the lab space.

Challenges

One challenge I faced during the lab was defining and validating meaningful physical or semantic units from raw sensor outputs. Another challenge occurred during early debugging, when some jumper wires on the breadboard were accidentally disconnected. At that time, no other team members were nearby. Since I don’t have any hardware background, I spent additional time reviewing breadboard layouts and understanding the correct placement of ground, 5v power, and sensor signal lines.

Team Work

Due to the weather conditions, we all work in a hybrid instead of in person. We have clearly divided our responsibilities across hardware integration and software development. While Mithun handled the main hardware work, I focused on the software side.

Plans

In the future of the project, I plan to continue supporting the software and control logic development of the robot, particularly for the dancing behavior, motion coordination and sensor transmission. I will take main responsibility for implementing and refining the control code while continuing to learn hardware-related concepts from my teammates since they are all from ECE and ME.

As the project progresses, I am also interested in discussing with the team potential ways to incorporate ml/dl into robots as more advanced motion strategies.

Joystick logical and showcase

```
String getMovementDirection(int x, int y) {
  int threshold = 20;

  if (abs(x) < threshold && abs(y) < threshold) {
    return "Stationary";
  }

  // Determine primary direction based on X and Y
  // Assuming positive X is forward, positive Y is right
  if (x > threshold && abs(y) < threshold) {
    return "Moving Forward";
  } else if (x < -threshold && abs(y) < threshold) {
    return "Moving Backward";
  } else if (abs(x) < threshold && y > threshold) {
    return "Moving Right";
  } else if (abs(x) < threshold && y < -threshold) {
    return "Moving Left";
  } else if (x > threshold && y > threshold) {
    return "Moving Forward-Right";
  } else if (x > threshold && y < -threshold) {
    return "Moving Forward-Left";
  } else if (x < -threshold && y > threshold) {
    return "Moving Backward-Right";
  } else if (x < -threshold && y < -threshold) {
    return "Moving Backward-Left";
  }

  return "Unknown";
}
```

```
Movement: Stationary
Movement: Moving Forward
Movement: Moving Backward
Movement: Moving Backward-Right
Paused: false   Paused: true
```

Orientation logical and showcase

```
String getCardinalDirection(float heading) {
  if ((heading >= 315 && heading <= 360) || (heading >= 0 && heading < 45)) {
    return "Facing North";
  } else if (heading >= 45 && heading < 135) {
    return "Facing East";
  } else if (heading >= 135 && heading < 225) {
    return "Facing South";
  } else if (heading >= 225 && heading < 315) {
    return "Facing West";
  }
  return "Unknown";
}
```

```
Direction: Facing North
Direction: Facing East
Direction: Facing South
Direction: Facing West
```

Potential meter logical and showcase

```
// Convert potentiometer value (0-1023) to speed level (1-10)
int getSpeedLevel(int potValue) {
  // Map 0-1023 to 1-10
  if (potValue < 100) return 1;
  else if (potValue < 200) return 2;
  else if (potValue < 300) return 3;
  else if (potValue < 400) return 4;
  else if (potValue < 500) return 5;
  else if (potValue < 600) return 6;
  else if (potValue < 700) return 7;
  else if (potValue < 800) return 8;
  else if (potValue < 900) return 9;
  else return 10;
}
```

```
Speed Level: 1

Speed Level: 10
```

Code

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>

#define ANALOG_X_PIN A1
#define ANALOG_Y_PIN A2
#define ANALOG_BUTTON_PIN A3

//Default values when axis not actioned
#define ANALOG_X_CORRECTION 128
#define ANALOG_Y_CORRECTION 128

struct button {
  byte pressed = 0;
};

struct analog {
  short x, y;

  button button;
};

double xPos = 0, yPos = 0, headingVel = 0;
uint16_t BNO055_SAMPLERATE_DELAY_MS = 10; //how often to read data from the board
uint16_t PRINT_DELAY_MS = 500; // how often to print the data
uint16_t printCount = 0; //counter to avoid printing every 10MS sample

//velocity = accel*dt (dt in seconds)
//position = 0.5*accel*dt^2
double ACCEL_VEL_TRANSITION =  (double)(BNO055_SAMPLERATE_DELAY_MS) / 1000.0;
double ACCEL_POS_TRANSITION = 0.5 * ACCEL_VEL_TRANSITION * ACCEL_VEL_TRANSITION;
double DEG_2_RAD = 0.01745329251; //trig functions require radians, BNO055 outputs
degrees

// Check I2C device address and correct line below (by default address is 0x29 or
0x28)
//                                   id, address
Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28);

void setup(void)
{
```

```
 // IMU
 Serial.begin(115200);
 while (!Serial) delay(10);  // wait for serial port to open!
 if (!bno.begin())
 {
   Serial.print("No BNO055 detected");
   while (1);
 }

 // Controller
 pinMode(ANALOG_BUTTON_PIN, INPUT_PULLUP);

 delay(1000);
}

// Convert heading angle (0-360) to cardinal direction
/*
0/360 North
90 East
180 South
270 West

+- 45
*/

String getCardinalDirection(float heading) {
 if ((heading >= 315 && heading <= 360) || (heading >= 0 && heading < 45)) {
   return "Facing North";
 } else if (heading >= 45 && heading < 135) {
   return "Facing East";
 } else if (heading >= 135 && heading < 225) {
   return "Facing South";
 } else if (heading >= 225 && heading < 315) {
   return "Facing West";
 }
 return "Unknown";
}

// Convert potentiometer value (0-1023) to speed level (1-10)
int getSpeedLevel(int potValue) {
 // Map 0-1023 to 1-10
 if (potValue < 100) return 1;
```

```
 else if (potValue < 200) return 2;
 else if (potValue < 300) return 3;
 else if (potValue < 400) return 4;
 else if (potValue < 500) return 5;
 else if (potValue < 600) return 6;
 else if (potValue < 700) return 7;
 else if (potValue < 800) return 8;
 else if (potValue < 900) return 9;
 else return 10;
}

// Convert joystick X/Y to movement direction
String getMovementDirection(int x, int y) {
 int threshold = 20;
  if (abs(x) < threshold && abs(y) < threshold) {
   return "Stationary";
 }
  // Determine primary direction based on X and Y
 // Assuming positive X is forward, positive Y is right
 if (x > threshold && abs(y) < threshold) {
   return "Moving Forward";
 } else if (x < -threshold && abs(y) < threshold) {
   return "Moving Backward";
 } else if (abs(x) < threshold && y > threshold) {
   return "Moving Right";
 } else if (abs(x) < threshold && y < -threshold) {
   return "Moving Left";
 } else if (x > threshold && y > threshold) {
   return "Moving Forward-Right";
 } else if (x > threshold && y < -threshold) {
   return "Moving Forward-Left";
 } else if (x < -threshold && y > threshold) {
   return "Moving Backward-Right";
 } else if (x < -threshold && y < -threshold) {
   return "Moving Backward-Left";
 }
  return "Unknown";
}

void loop(void)
{
 // IMU - Angle
```

```
unsigned long tStart = micros();
sensors_event_t orientationData , linearAccelData;
bno.getEvent(&orientationData, Adafruit_BNO055::VECTOR_EULER);
//  bno.getEvent(&angVelData, Adafruit_BNO055::VECTOR_GYROSCOPE);
bno.getEvent(&linearAccelData, Adafruit_BNO055::VECTOR_LINEARACCEL);

xPos = xPos + ACCEL_POS_TRANSITION * linearAccelData.acceleration.x;
yPos = yPos + ACCEL_POS_TRANSITION * linearAccelData.acceleration.y;

// velocity of sensor in the direction it's facing
headingVel = ACCEL_VEL_TRANSITION * linearAccelData.acceleration.x / cos(DEG_2_RAD *
orientationData.orientation.x);

int sensorValue = analogRead(A0);

analog analog;

analog.x = readAnalogAxisLevel(ANALOG_X_PIN) - ANALOG_X_CORRECTION;
analog.y = readAnalogAxisLevel(ANALOG_Y_PIN) - ANALOG_Y_CORRECTION;

analog.button.pressed = isAnalogButtonPressed(ANALOG_BUTTON_PIN);

if (printCount * BNO055_SAMPLERATE_DELAY_MS >= PRINT_DELAY_MS) {
  //enough iterations have passed that we can print the latest data

  // IMU direction - converted to cardinal direction
  Serial.print("Direction: ");
  Serial.println(getCardinalDirection(orientationData.orientation.x));

  // Potentiometer - converted to speed level
  Serial.print("Speed Level: ");
  Serial.println(getSpeedLevel(sensorValue));
   // Joystick - converted to movement direction
  Serial.print("Movement: ");
  Serial.println(getMovementDirection(analog.x, analog.y));

  // Button - converted to pause state
  Serial.print("Paused: ");
  if (analog.button.pressed) {
    Serial.println("true");
  } else {
    Serial.println("false");
```

```
    }

    Serial.println("---"); // Separator for readability
    printCount = 0;
    delay(1000);
  }
  else {
    printCount = printCount + 1;
  }
}

byte readAnalogAxisLevel(int pin)
{
  return map(analogRead(pin), 0, 1023, 0, 255);
}

bool isAnalogButtonPressed(int pin)
{
  return digitalRead(pin) == 0;
}
```